D:\博士科研\实践：**scPRINT-Large Cell Model for scRNAseq data\model.py**

```python
# from scprint.base.base_model import BaseModel
import copy
import datetime
import os
from functools import partial

# from galore_torch import GaLoreAdamW
from math import factorial
from pathlib import Path
from typing import Dict, Optional

import lightning as L
import pandas as pd
import torch
import torch.distributed
from huggingface_hub import PyTorchModelHubMixin
from lightning.pytorch.callbacks.lr_finder import LearningRateFinder
from lightning.pytorch.tuner.lr_finder import _LRCallback
from scipy.sparse import load_npz
from simpler_flash import FlashTransformer
from torch import Tensor, nn, optim

# from .linear_transformer import FastTransformerEncoderWrapper as FastTransformer
from . import decoders, encoders, fsq, loss, utils
from .loss import grad_reverse
from .utils import WeightedMasker, simple_masker

FILEDIR = os.path.dirname(os.path.realpath(__file__))


def is_interactive():
    import __main__ as main

    return not hasattr(main, "__file__")


class scPrint(L.LightningModule, PyTorchModelHubMixin):
    def __init__(
        self,
        genes: list,
        organisms: list = ["NCBITaxon:9606"],
        precpt_gene_emb: Optional[str] = None,
        gene_pos_enc: Optional[list] = None,
        normalization: str = "sum",
        d_model: int = 512,
        nhead: int = 8,
        attn_bias: str = "none",
        d_hid: int = 512,
        edge_dim: int = 12,
        nlayers: int = 6,
        expr_encoder_layers: int = 2,
        layers_cls: list[int] = [],
        classes: Dict[str, int] = {},
        labels_hierarchy: Dict[str, Dict[int, list[int]]] = {},
        dropout: float = 0.1,
        transformer: str = "fast",
        expr_emb_style: str = "continuous",  # "binned_pos", "cont_pos"
        domain_spec_batchnorm: str = "None",
        n_input_bins: int = 0,
        num_batch_labels: int = 0,
        mvc_decoder: str = "None",
        pred_embedding: list[str] = [],
        cell_emb_style: str = "cls",
        cell_specific_blocks: bool = False,
        depth_atinput: bool = True,
        freeze_embeddings: bool = True,
        label_decoders: Optional[Dict[str, Dict[int, str]]] = None,
        zinb: bool = True,
        lr: float = 0.0001,
        compress_class_dim: Optional[Dict[str, int]] = None,
        **flash_attention_kwargs,
    ):
```

```python
        """
        scPRINT transformer for single cell biology and the inference of Gene Regulatory networks

        Args:
            genes (list): List of gene names the model will work with.
            precpt_gene_emb (np.array, optional): Gene embeddings of size (len(genes), d_model). Should be in the same order as the genes. Defaults to None.
            gene_pos_enc (list, optional): Gene position encoding of the same size as genes. Provides a location value for each gene in genes. Defaults to None.
            d_model (int, optional): Dimension of the model. Defaults to 512.
            nhead (int, optional): Number of heads in the multihead attention models. Defaults to 8.
            d_hid (int, optional): Dimension of the feedforward network model. Defaults to 512.
            nlayers (int, optional): Number of layers in the transformer model. Defaults to 6.
            expr_encoder_layers (int, optional): Number of layers in the expression encoder. Defaults to 2.
            layers_cls (list[int], optional): List specifying the number of layers in the classifier. Defaults to [].
            classes (Dict[str, int], optional): Classes to predict with the number of classes for each. Defaults to {}.
            labels_hierarchy (Dict[str, Dict[int, list[int]]], optional): Class hierarchy for classes with hierarchical classes. Defaults to {}.
            dropout (float, optional): Dropout value. Defaults to 0.2.
            transformer (str, optional): Transformer type to use. One of "linear", "flash", "flashsparse", "scprint". Defaults to "fast".
            domain_spec_batchnorm (str, optional): Whether to apply domain-specific batch normalization. Defaults to "None".
            expr_emb_style (str, optional): Style of input embedding. One of "continuous", "binned_pos", "cont_pos". Defaults to "continuous".
            mvc_decoder (str, optional): Style of MVC decoder. One of "None", "inner product", "concat query", "sum query". Defaults to "None".
            pred_embedding (list[str], optional): List of classes to use for plotting embeddings. Defaults to [].
            cell_emb_style (str, optional): Style of cell embedding. One of "cls", "avg-pool", "w-pool". Defaults to "cls".
            freeze_embeddings (bool, optional): Whether to freeze the embeddings during training. Defaults to True.
            label_decoders (Optional[Dict[str, Dict[int, str]]], optional): Label decoders to use for plotting the UMAP during validations. Defaults to None.
            zinb (bool, optional): Whether to use Zero-Inflated Negative Binomial distribution. Defaults to True.
            **flash_attention_kwargs (dict): Additional keyword arguments for the model. see @flashformer.py

        Notes:
            for other parameters of the model that are not part of its class definition, see @trainer.trainer.py

        Raises:
            ValueError: If the expr_emb_style is not one of "continuous", "binned_pos", "cont_pos".
        """
        super().__init__()
        self.save_hyperparameters()
        # training flags
        self.do_denoise = True
        self.noise = [0.6]
        self.do_cce = False
        self.cce_temp = 0.2
        self.lr = 0.0001
        self.cce_scale = 0.1
        self.do_ecs = False
        self.ecs_threshold = 0.4
        self.ecs_scale = 0.1
        self.do_mvc = False
        self.mvc_scale = 1.0
        self.class_embd_diss_scale = 0.1
        self.do_adv_cls = False
        self.adv_class_scale = 0.1
        self.do_cls = False
        self.mean_attn_tot = None
        self.mean_attn_tot_c = 0
        self.do_adv_batch = False
        self.run_full_forward = True
        self.class_scale = 1
        self.zinb_and_mse = False
        self.do_next_tp = False
        self.do_generate = False
        self.var_context_length = False
        self.depth_atinput = depth_atinput
        self.mask_ratio = []
        self.warmup_duration = 500
        self.weight_decay = 0.01
        self.optim = "adamW"
        self.fused_adam = False
```

```
        self.lr_reduce_patience = 2
        self.lr_reduce_factor = 0.6
        self.test_every = 20
        self.lr_reduce_monitor = "val_loss"
        self.name = ""
        self.lrfinder_steps = 0
        self.doplot = True
        self.get_attention_layer = []
        self.embs = None
        self.pred_log_adata = True
        self.attn = utils.Attention(
            len(genes),
            additional_tokens=(
                len(classes) + (2 if self.depth_atinput else 1)
                if not cell_specific_blocks
                else 0
            ),
        )
        self.tf_masker = WeightedMasker(genes, inv_weight=0.05)
        self.predict_depth_mult = 3
        self.predict_mode = "none"
        self.keep_all_cls_pred = False
        self.cell_separation = True
        # should be stored somehow
        self.d_model = d_model
        self.normalization = normalization
        self.organisms = organisms
        self.edge_dim = edge_dim
        self.attn_bias = attn_bias
        self.nlayers = nlayers
        self.gene_pos_enc = gene_pos_enc
        self.mvc_decoder = mvc_decoder
        self.domain_spec_batchnorm = domain_spec_batchnorm
        # need to store
        self.n_input_bins = n_input_bins
        self.transformer = transformer
        self.label_counts = classes
        self.classes = list(classes.keys())

        if cell_emb_style not in ["cls", "avg-pool", "w-pool"]:
            raise ValueError(f"Unknown cell_emb_style: {cell_emb_style}")
        self.cell_emb_style = cell_emb_style

        self.label_decoders = label_decoders
        self.pred_embedding = pred_embedding
        self.genes = genes
        self.vocab = {i: n for i, n in enumerate(genes)}
        self.expr_emb_style = expr_emb_style
        if self.expr_emb_style not in ["category", "continuous", "none"]:
            raise ValueError(
                f"expr_emb_style should be one of category, continuous, scaling, "
                f"got {expr_emb_style}"
            )
        self.labels_hierarchy = labels_hierarchy
        self.hparams["labels_hierarchy"] = self.labels_hierarchy
        self.hparams["classes"] = self.classes
        self.hparams["label_decoders"] = self.label_decoders
        self.hparams["label_counts"] = self.label_counts
        self.hparams["gene_pos_enc"] = self.gene_pos_enc
        self.hparams["genes"] = self.genes

        self.mat_labels_hierarchy = {}
        for k, v in labels_hierarchy.items():
            tens = torch.zeros((len(v), classes[k]))
            for k2, v2 in v.items():
                tens[k2 - classes[k], v2] = 1
            self.mat_labels_hierarchy[k] = tens.to(bool)

        # encoder
        # gene encoder
        if precpt_gene_emb is not None:
            embeddings = pd.read_parquet(precpt_gene_emb).loc[self.genes]
            if len(embeddings) == 0:
                raise ValueError(
```

```
                    f"the gene embeddings file {precpt_gene_emb} does not contain any of the genes given to the model"
                )
            elif len(embeddings) < len(self.genes):
                print(
                    "Warning: only a subset of the genes available in the embeddings file."
                )
                print("number of genes: ", len(embeddings))
            sembeddings = torch.nn.AdaptiveAvgPool1d(d_model)(
                torch.tensor(embeddings.values)
            )

            self.gene_encoder = encoders.GeneEncoder(
                len(self.vocab), d_model, weights=sembeddings, freeze=freeze_embeddings
            )
        else:
            self.gene_encoder = encoders.GeneEncoder(len(self.vocab), d_model)

        # Value Encoder, NOTE: the scaling style is also handled in _encode method
        if expr_emb_style in ["continuous", "full_pos"]:
            self.expr_encoder = encoders.ContinuousValueEncoder(
                d_model, dropout, layers=expr_encoder_layers
            )
        elif expr_emb_style == "binned_pos":
            assert n_input_bins > 0
            self.expr_encoder = encoders.CategoryValueEncoder(n_input_bins, d_model)
        else:
            self.expr_encoder = torch.nn.Identity()

        # Positional Encoding
        if self.gene_pos_enc is not None:
            max_len = max(gene_pos_enc)
            token_to_pos = {token: pos for token, pos in enumerate(self.gene_pos_enc)}
            self.pos_encoder = encoders.PositionalEncoding(
                d_model, max_len=max_len, token_to_pos=token_to_pos
            )

        self.cell_embs_count = len(self.classes) + (2 if self.depth_atinput else 1)
        # Class Encoder
        # always have [base_cell_emb, time_embedding, depth_embedding] + any other class info
        # base cell embedding will store other cell specific information
        self.class_encoder = encoders.CategoryValueEncoder(
            self.cell_embs_count - (1 if self.depth_atinput else 0), d_model
        )
        # self.time_encoder = encoders.ContinuousValueEncoder(d_model, dropout)
        self.depth_encoder = encoders.ContinuousValueEncoder(
            d_model, dropout, layers=expr_encoder_layers
        )

        # compute tensor for mat_labels_hierarchy
        for i in ["strict_loading", "optim", "weight_decay", "lr"]:
            if i in flash_attention_kwargs:
                flash_attention_kwargs.pop(i)
        # Transformer
        # Linear
        if transformer == "linear":
            # linear transformer using the fast transformer package
            # self.transformer = FastTransformerEncoder(
            #    d_model, nhead, d_hid, nlayers, dropout, "linear"
            # )
            raise NotImplementedError("Linear transformer is not implemented")
        # regular or flash
        else:
            self.transformer = FlashTransformer(
                d_model=d_model,
                nhead=nhead,
                dropout=dropout,
                nlayers=nlayers,
                cross_attn=cell_specific_blocks,
                use_flash_attn=(transformer == "flash"),
                **flash_attention_kwargs,
            )
        if cell_specific_blocks:
            self.cell_transformer = FlashTransformer(
                d_model=d_model,
```

```
                nhead=nhead,
                nlayers=6,
                dropout=dropout,
                cross_attn=True,
                use_flash_attn=(transformer == "flash"),
                **flash_attention_kwargs,
            )
        else:
            self.cell_transformer = None

        # decoders
        # expression
        self.expr_decoder = decoders.ExprDecoder(
            d_model,
            nfirst_tokens_to_skip=self.cell_embs_count,
            dropout=dropout,
            zinb=zinb,
            use_depth=not self.depth_atinput,
        )
        # cls decoder
        self.cls_decoders = torch.nn.ModuleDict()
        # should be a very simple classifier for most things
        # (maybe scale with the number of classes) should be 1 layer...
        for clss, n_cls in classes.items():
            self.cls_decoders[clss] = decoders.ClsDecoder(
                d_model, n_cls, layers=layers_cls, dropout=dropout
            )

        # Batch effect correction via adversarial training on batch classes
        if num_batch_labels > 0:
            self.grad_reverse_discriminator_loss = loss.AdversarialDiscriminatorLoss(
                d_model,
                n_cls=num_batch_labels,
            )
        else:
            self.grad_reverse_discriminator_loss = None

        # expression decoder from batch embbedding
        if mvc_decoder != "None":
            self.mvc_decoder = decoders.MVCDecoder(
                d_model,
                arch_style=mvc_decoder,
                zinb=zinb,
            )
        else:
            self.mvc_decoder = None

        self.apply(
            partial(
                utils._init_weights,
                n_layer=nlayers,
            )
        )
        for i, dec in self.cls_decoders.items():
            torch.nn.init.constant_(dec.out_layer.bias, -0.13)

        if compress_class_dim is not None:
            self.bottleneck_mlps = torch.nn.ModuleDict()
            for k, v in compress_class_dim.items():
                self.bottleneck_mlps[k] = fsq.FSQ(levels=[2] * v, dim=self.d_model)
        else:
            self.bottleneck_mlps = None

    def on_load_checkpoint(self, checkpoints):
        for name, clss in self.cls_decoders.items():
            size = checkpoints["state_dict"][
                "cls_decoders." + name + ".out_layer.bias"
            ].shape[0]
            if size != clss.out_layer.bias.shape[0]:
                self.cls_decoders[name].out_layer = torch.nn.Linear(
                    clss.out_layer.weight.shape[1], size
                )
        size = checkpoints["state_dict"]["class_encoder.embedding.weight"].shape[0]
        if size != self.class_encoder.embedding.weight.shape[0]:
            self.class_encoder = encoders.CategoryValueEncoder(size, self.d_model)
```

```
            self.cell_embs_count = size
            print("changing size, could lead to issues")
        size = checkpoints["state_dict"][
            "grad_reverse_discriminator_loss.out_layer.bias"
        ].shape[0]
        # we won't use it but still need to take care of it. for now will still add it to the model
        if size != self.grad_reverse_discriminator_loss.out_layer.bias.shape[0]:
            self.grad_reverse_discriminator_loss = loss.AdversarialDiscriminatorLoss(
                self.d_model,
                n_cls=size,
            )
            print(
                "the discriminator for batch effect correction has been resized\
                and re-initiliazed. It will start from scratch during this training if "
            )

        # if len(checkpoints["state_dict"]["pos_encoder.pe"].shape) == 3:
        #    self.pos_encoder.pe = checkpoints["state_dict"]["pos_encoder.pe"].squeeze(1)

        self.normalization = checkpoints["hyper_parameters"]["normalization"]
        if "classes" in checkpoints["hyper_parameters"]:
            if self.classes != checkpoints["hyper_parameters"]["classes"]:
                print("changing the number of classes, could lead to issues")

            if "label_counts" in checkpoints["hyper_parameters"]:
                self.label_counts = checkpoints["hyper_parameters"]["label_counts"]
                self.classes = checkpoints["hyper_parameters"]["classes"]
            else:
                self.label_counts = checkpoints["hyper_parameters"]["classes"]
                self.classes = list(self.label_counts.keys())
            self.label_decoders = checkpoints["hyper_parameters"]["label_decoders"]
            self.labels_hierarchy = checkpoints["hyper_parameters"]["labels_hierarchy"]
            for k, v in self.labels_hierarchy.items():
                tens = torch.zeros((len(v), self.label_counts[k]))
                for k2, v2 in v.items():
                    tens[k2 - self.label_counts[k], v2] = 1
                self.mat_labels_hierarchy[k] = tens.to(bool)
        if "gene_pos_enc" in checkpoints["hyper_parameters"]:
            if self.genes != checkpoints["hyper_parameters"]["genes"]:
                raise ValueError(
                    "Genes or their ordering have changed in the dataloader compared to last time, the model will likely misbehave!"
                )
            if self.gene_pos_enc != checkpoints["hyper_parameters"]["gene_pos_enc"]:
                print(
                    "Gene position encoding has changed in the dataloader compared to last time, be careful!"
                )
        mencoders = {}
        try:
            if self.trainer.datamodule.decoders != self.label_decoders:
                # if we don't have the same decoders, we need to update the one on the datamodule side
                for k, v in checkpoints["hyper_parameters"]["label_decoders"].items():
                    mencoders[k] = {va: ke for ke, va in v.items()}
                self.trainer.datamodule.dataset.mapped_dataset.encoders = mencoders
                if (
                    self.trainer.datamodule.kwargs["collate_fn"].organism_name
                    in mencoders
                ):
                    self.trainer.datamodule.kwargs["collate_fn"]._setup(
                        org_to_id=mencoders[
                            self.trainer.datamodule.kwargs["collate_fn"].organism_name
                        ],
                        valid_genes=self.genes,
                    )
            os.environ["MY_SLURM_RESTART_COUNT"] = str(
                int(os.getenv("SLURM_RESTART_COUNT", 0))
                + 1
                + int(os.getenv("MY_SLURM_RESTART_COUNT", 0))
            )
        except RuntimeError as e:
            if "scPrint is not attached to a `Trainer`." in str(e):
                print("RuntimeError caught: scPrint is not attached to a `Trainer`.")
        if not is_interactive():
            self.save_hyperparameters()

```

```python
    def _encoder(
        self,
        gene_pos: Tensor,
        expression: Optional[Tensor] = None,
        mask: Optional[Tensor] = None,
        req_depth: Optional[Tensor] = None,
        timepoint: Optional[Tensor] = None,
        cell_embs: Optional[Tensor] = None,  # (minibatch, n_labels, embsize)
    ):
        """
        _encode given inputs to the model encode into embeddings.

        Args:
            @see self.forward()

        Returns:
            Tensor: the encoded data
        """
        enc = self.gene_encoder(gene_pos)  # (minibatch, seq_len, embsize)
        self.cur_gene_token_embs = enc.clone()

        if expression is not None:
            if self.normalization == "sum":
                norm_expr = expression / expression.sum(1).unsqueeze(1)

            elif self.normalization == "log":
                norm_expr = torch.log2(1 + expression)
            else:
                raise ValueError(f"Unknown normalization: {self.normalization}")
            enc.add_(self.expr_encoder(norm_expr, mask))
        if self.gene_pos_enc:
            enc.add_(self.pos_encoder(gene_pos))
        if cell_embs is None:
            cell_embs = self.class_encoder(
                torch.arange(
                    self.cell_embs_count - (1 if self.depth_atinput else 0),
                    device=expression.device,
                ).repeat(expression.shape[0], 1)
            )
        if timepoint is not None:
            pass
            # cell_embs[:, 2, :] = self.time_encoder(timepoint)
        if req_depth is not None:
            # cell_embs = cell_embs.clone()
            # cell_embs[:, 1, :] += self.depth_encoder(torch.log2(1 + req_depth))
            depth_encoded = self.depth_encoder(torch.log2(1 + req_depth)).unsqueeze(1)
            cell_embs = torch.cat(
                (cell_embs[:, :1, :], depth_encoded, cell_embs[:, 1:, :]), dim=1
            )
        return torch.cat([cell_embs, enc], dim=1)  # self.norm_and_dropout(enc)
        # we already apply prenorm & dropout  # (minibatch, seq_len, embsize)

    def _decoder(
        self,
        transformer_output,
        depth_mult,
        get_gene_emb=False,
        do_sample=False,
        do_mvc=False,
        do_class=False,
        req_depth: Optional[Tensor] = None,
    ):
        """
        _decoder given the transformer output, decode into the final output.

        Args:
            @see self.forward()

        Returns:
            dict: the output of the model
        """
        if req_depth is not None:
            req_depth = torch.log2(1 + req_depth)
        output = self.expr_decoder(transformer_output, req_depth)

```

```python
        output["mean"] = depth_mult.unsqueeze(1) * output["mean"]
        if do_sample:
            pass

        output["cell_embs"] = self.get_cell_embs(transformer_output)
        if self.bottleneck_mlps is not None:
            for i, clsname in enumerate(self.classes):
                loc = i + (2 if self.depth_atinput else 1)
                output["cell_embs"][:, loc, :] = self.bottleneck_mlps[clsname](
                    output["cell_embs"][:, loc, :]
                )[0]
        output["cell_emb"] = torch.mean(output["cell_embs"].clone(), dim=1)
        if len(self.classes) > 0 and do_class:
            output.update(
                {
                    "cls_output_" + clsname: self.cls_decoders[clsname](
                        output["cell_embs"][
                            :, i + (2 if self.depth_atinput else 1), :
                        ]  # the first elem is the base cell embedding
                    )
                    for i, clsname in enumerate(self.classes)
                }
            )  # (minibatch, n_cls)
        if do_mvc:
            output.update(
                self.mvc_decoder(output["cell_emb"], self.cur_gene_token_embs)
            )
            output["mvc_mean"] = (
                depth_mult.unsqueeze(1) * output["mvc_mean"]
            )  # (minibatch, seq_len)

        if get_gene_emb:
            output["gene_embedding"] = transformer_output[
                :, self.cell_embs_count :, :
            ]  # (minibatch, seq_len, embsize)
        return output

    def forward(
        self,
        gene_pos: Tensor,
        expression: Optional[Tensor] = None,
        mask: Optional[Tensor] = None,
        req_depth: Optional[Tensor] = None,
        timepoint: Optional[Tensor] = None,  # (new_minibatch_of_nxt_cells,)
        get_gene_emb: bool = False,
        depth_mult: Optional[Tensor] = None,
        do_sample: bool = False,
        do_mvc: bool = False,
        do_class: bool = False,
        get_attention_layer: list = [],
    ):
        """
        forward also called on self(), a full forward pass on the model

        Args:
            gene_pos (Tensor): A tensor of shape (minibatch, seq_len)
                representing the genes used for each cell in the minibatch.
            expression (Tensor, optional): A tensor of shape (minibatch, seq_len)
                representing the expression levels of genes in the minibatch. Defaults to None.
            mask (Tensor, optional): A tensor of shape (minibatch, seq_len)
                used to mask certain elements in the sequence during the forward pass. Defaults to None.
            req_depth (Tensor, optional): A tensor of shape (minibatch,)
                representing the full depth of each sequence in the minibatch. Defaults to None.
            depth_mult (Tensor, optional): A tensor of shape (minibatch,)
                representing the depth multiplier for each sequence in the minibatch. Defaults to None.
            timepoint (Tensor, optional): A tensor of shape (minibatch,)
                representing the timepoint associated with each sequence in the minibatch. Defaults to None.
            get_gene_emb (bool, optional): A flag indicating whether to return the gene embeddings.
                If True, the gene embeddings are included in the output. Defaults to False.
            do_sample (bool, optional): A flag indicating whether to sample the expression levels.
                If True, the expression levels are sampled during the forward pass. Defaults to False.
            get_attention_layer (list, optional): A list indicating which attention layers to return.
                If not empty, the specified attention layers are included in the output. Defaults to [].

        Returns:
```

```
            dict of output Tensors: A dictionary containing the output tensors from the forward pass.
                The keys of the dictionary depend on the input flags (get_gene_emb, do_sample, get_attention_layer).
                at minima, the dictionary codntains the following:
                - "mean": the mean expression levels
                - "zero_logits": the logits for zero-inflated expression levels
                - "disp": the dispersion parameter
                - "cell_embs": the cell embeddings per class
                - "cell_emb": the main cell embedding
                - "cls_output": the output of the classifier
        """
        encoding = self._encoder(
            gene_pos,
            expression,
            mask,
            req_depth=req_depth if self.depth_atinput else None,
            timepoint=timepoint,
        )
        if self.attn_bias != "none":
            if not hasattr(self, "nbias"):
                bias_path = os.path.join(
                    Path(FILEDIR).parent.parent, "data", "bias_sparse.npz"
                )
                self.nbias = torch.Tensor(load_npz(bias_path).todense()).to(
                    device=gene_pos.device, dtype=torch.float16
                )
            num = self.cell_embs_count if not self.cell_transformer else 0
            bias = torch.zeros(
                (
                    gene_pos.shape[0],
                    gene_pos.shape[1] + num,
                    gene_pos.shape[1] + num,
                ),
                device=gene_pos.device,
                dtype=torch.float16,
            )
            # fade slowly through the iterations
            fade_factor = 400 / (400 + self.trainer.global_step)
            # bias[:, num:, :num] = -10_000  # do not pay attention to the cls embeddings
            bias[:, num:, num:] = (
                self.nbias[gene_pos[:, :, None], gene_pos[:, None, :]] * fade_factor
            )
        if self.cell_transformer:
            cell_encoding = encoding[:, : self.cell_embs_count, :]
            encoding = encoding[:, self.cell_embs_count :, :]
        transformer_output = self.transformer(
            encoding,
            return_qkv=get_attention_layer,
            bias=bias if self.attn_bias != "none" else None,
            bias_layer=list(range(self.nlayers - 1)),
        )
        if len(get_attention_layer) > 0:
            transformer_output, qkvs = transformer_output
        if self.cell_transformer:
            cell_output = self.cell_transformer(cell_encoding, x_kv=transformer_output)
            transformer_output = torch.cat([cell_output, transformer_output], dim=1)
        # if not provided we will mult by the current expression sum
        depth_mult = expression.sum(1) if depth_mult is None else depth_mult
        res = self._decoder(
            transformer_output,
            depth_mult,
            get_gene_emb,
            do_sample,
            do_mvc,
            do_class,
            req_depth=req_depth if not self.depth_atinput else None,
        )
        return (res, qkvs) if len(get_attention_layer) > 0 else res

    def configure_optimizers(self):
        """@see pl.LightningModule"""
        # https://pytorch.org/docs/stable/generated/torch.optim.Adam.html#torch.optim.Adam
        # not working because of poor weight decay implem
        if self.optim == "adam":
            optimizer = optim.Adam(
```

```
                self.parameters(),
                lr=self.hparams.lr,
                betas=(0.9, 0.999),
                eps=1e-08,
                weight_decay=self.weight_decay,
                amsgrad=False,
                fused=self.fused_adam,
            )
        elif self.optim == "adamW":
            optimizer = optim.AdamW(
                self.parameters(),
                lr=self.hparams.lr,
                betas=(0.9, 0.999),
                eps=1e-08,
                weight_decay=self.weight_decay,
                amsgrad=False,
                fused=self.fused_adam,
            )
        elif self.optim == "galore":
            raise NotImplementedError("Galore optimizer not implemented")
            # param_groups = [
            #    {
            #        "params": [
            #            v for k, v in self.named_parameters() if "transformer" not in k
            #        ]
            #    },
            #    {
            #        "params": [
            #            v for k, v in self.named_parameters() if "transformer" in k
            #        ],
            #        "rank": 128,
            #        "update_proj_gap": 200,
            #        "scale": 0.25,
            #        "proj_type": "std",
            #    },
            # ]
            # optimizer = GaLoreAdamW(param_groups, lr=self.hparams.lr)
        else:
            raise ValueError(f"Unknown optimizer: {self.optim}")
        if self.lr_reduce_monitor is None:
            print("no lr reduce factor")
            return [optimizer]
        lr_scheduler = optim.lr_scheduler.ReduceLROnPlateau(
            optimizer,
            mode="min",
            patience=self.lr_reduce_patience,
            factor=self.lr_reduce_factor,
            verbose=True,
        )
        lr_dict = {
            "scheduler": lr_scheduler,
            # The unit of the scheduler's step size, could also be 'step'.
            # 'epoch' updates the scheduler on epoch end whereas 'step'
            # updates it after a optimizer update.
            "interval": "epoch",
            # How many epochs/steps should pass between calls to
            # `scheduler.step()`. 1 corresponds to updating the learning
            # rate after every epoch/step.
            "frequency": 1,
            # Metric to to monitor for schedulers like `ReduceLROnPlateau`
            "monitor": self.lr_reduce_monitor,
        }
        self.lrfinder_steps = 0
        for val in self.trainer.callbacks:
            if type(val) is _LRCallback:
                self.lrfinder_steps = val.num_training
            if type(val) is LearningRateFinder:
                self.lrfinder_steps = val._num_training_steps
        return [optimizer], [lr_dict]

    def on_fit_start(self):
        """@see pl.LightningModule"""
        if type(self.transformer) is FlashTransformer:
            for encoder_layers in self.transformer.blocks:
                encoder_layers.set_seq_parallel(True)
```

```python
        for k, v in self.mat_labels_hierarchy.items():
            self.mat_labels_hierarchy[k] = v.to(self.device)

    def training_step(
        self,
        batch: Dict[str, Tensor],
        batch_idx,
    ):
        """
        training_step defines the train loop. It is independent of forward

        @see pl.LightningModule

        Returns:
            _type_: _description_
        """
        total_loss, losses = self._full_training(
            batch=batch,
            do_denoise=self.do_denoise,
            noise=self.noise,
            do_next_tp=self.do_next_tp,
            do_cce=self.do_cce,
            cce_temp=self.cce_temp,
            do_ecs=self.do_ecs,
            do_mvc=self.do_mvc,
            do_adv_cls=self.do_adv_cls,
            do_adv_batch=self.do_adv_batch,
            do_cls=self.do_cls,
            do_generate=self.do_generate,
            run_full_forward=self.run_full_forward,
            mask_ratio=self.mask_ratio,
        )

        self.log("train_loss", total_loss, prog_bar=True, sync_dist=True)
        self.log_dict(losses, prog_bar=True, sync_dist=True)
        return total_loss

    def _full_training(
        self,
        batch: Dict[str, Tensor],
        do_denoise: bool = False,
        noise: list[float] = [],
        do_next_tp: bool = False,
        do_cce: bool = False,
        cce_temp: float = 0.5,
        do_ecs: bool = False,
        do_mvc: bool = False,
        do_adv_cls: bool = False,
        do_adv_batch: bool = False,
        do_cls: bool = False,
        do_generate: bool = False,
        run_full_forward: bool = True,
        mask_ratio: list[float] = [0.15],
    ):
        """
        _full_training implement the trainng steps: forward (multiple sometimes), loss

        Args:
            batch (dict[Tensors]): A dictionary containing tensors for the training batch:
                - "x": the expression levels of genes in the minibatch
                - "genes": the genes used for each cell in the minibatch
                - "class": the class to predict for each cell
                - "depth": the full depth of each cell in the minibatch
            do_denoise (bool, optional): A flag to indicate whether to perform denoising. Defaults to False.
            noise (list[float], optional): A list of noise levels to be used in denoising. Defaults to [].
            do_next_tp (bool, optional): A flag to indicate whether to perform next time point prediction. Defaults to False.
            do_cce (bool, optional): A flag to indicate whether to perform cross-categorical entropy. Defaults to False.
            cce_temp (float, optional): The similarity threshold for cross-categorical entropy. Defaults to 0.5.
            do_ecs (bool, optional): A flag to indicate whether to perform elastic cell similarity. Defaults to False.
            do_mvc (bool, optional): A flag to indicate whether to perform multi-view coding. Defaults to False.
            do_adv_cls (bool, optional): A flag to indicate whether to perform adversarial classification. Defaults to False.
```

```python
            do_generate (bool, optional): A flag to indicate whether to perform data generation. Defaults to False.
            mask_ratio (list, optional): A list of mask ratios to be used in the training. Defaults to [0.15].

        Returns:
            loss, losses: the total loss as float and the individual losses as dict
        """
        if type(mask_ratio) is not list:
            mask_ratio = [mask_ratio]
        # dynamically change the context length every 5 steps
        if self.var_context_length and self.trainer.global_step % 5 == 0:
            context_length = torch.randint(400, batch["x"].shape[1], (1,)).item()
        else:
            context_length = batch["x"].shape[1]
        expression = batch["x"][:, :context_length]
        gene_pos = batch["genes"][:, :context_length]
        total_count = batch["depth"]
        clss = batch.get("class", None)
        batch_idx = batch.get("dataset", None)

        total_loss = 0
        losses = {}
        cell_embs = []
        if run_full_forward:
            output = self.forward(
                gene_pos,
                expression,
                mask=None,
                req_depth=total_count,
                do_mvc=do_mvc,
                do_class=do_cls,
            )
            if "disp" in output:
                output.pop("disp")
            if "zero_logits" in output:
                output.pop("zero_logits")
            if "mean" in output:
                output.pop("mean")
            l, tot = self._compute_loss(
                output,
                expression,
                clss,
                batch_idx,
                do_ecs,
                do_adv_cls & do_cls,
                do_adv_batch & do_cls,
            )
            cell_embs.append(output["cell_emb"].clone())
            full_cell_embs = output["cell_embs"].clone()
            total_loss += tot
            losses.update({"full_forward_" + k: v for k, v in l.items()})
            do_mvc = False
            do_cls = False

        for i in mask_ratio:
            # do noise and mask
            if do_denoise:
                expr = utils.downsample_profile(expression, dropout=0.5, randsamp=True)
            else:
                expr = expression
            if i == "TF":
                mask = self.tf_masker(
                    ids=gene_pos,
                    mask_ratio=0.3,
                ).to(gene_pos.device)
            else:
                mask = simple_masker(
                    shape=gene_pos.shape,
                    mask_ratio=i,
                ).to(gene_pos.device)
            output = self.forward(
                gene_pos,
                expression=expr,
                mask=mask,
                req_depth=expr.sum(1),
```

```
                do_mvc=do_mvc,
                do_class=do_cls,
            )
            l, tot = self._compute_loss(
                output,
                expr,
                clss,
                batch_idx,
                do_ecs,
                do_adv_cls & do_cls,
                do_adv_batch & do_cls,
                do_mse=self.zinb_and_mse,
            )
            # we only want to do them once
            do_mvc = False
            do_cls = False

            cell_embs.append(output["cell_emb"].clone())
            total_loss += tot
            pct = str(int(i * 100)) + "%_" if i != "TF" else "TF_"
            losses.update({"mask_" + pct + k: v for k, v in l.items()})
        # TASK 3. denoising
        if do_denoise:
            for i in noise:
                expr = utils.downsample_profile(expression, dropout=i)
                output = self.forward(
                    gene_pos,
                    expression=expr,
                    mask=None,
                    depth_mult=expression.sum(1),
                    req_depth=total_count,
                    do_mvc=do_mvc,
                    do_class=do_cls,
                )
                l, tot = self._compute_loss(
                    output,
                    expression,
                    clss,
                    batch_idx,
                    do_ecs,
                    do_adv_cls & do_cls,
                    do_adv_batch & do_cls,
                    do_mse=self.zinb_and_mse,
                )
                do_mvc = False
                do_cls = False

                cell_embs.append(output["cell_emb"].clone())
                total_loss += tot
                losses.update(
                    {"denoise_" + str(int(i * 100)) + "%_" + k: v for k, v in l.items()}
                )
                # make sure that the cell embedding stay the same even if the expression is decreased

        # TASK 6. expression generation
        if do_generate:
            output = self._generate(
                cell_embs=output["cell_embs"]
                if not run_full_forward
                else full_cell_embs,
                gene_pos=gene_pos,
                depth_mult=expression.sum(1),
                req_depth=total_count,
                do_mvc=do_mvc,
                do_class=do_cls,
            )
            if "cell_emb" in output:
                cell_embs.append(output["cell_emb"].clone())
            l, tloss = self._compute_loss(
                output,
                expression,
                clss,
                batch_idx,
                ("cell_emb" in output) and do_ecs,
                do_adv_cls & do_cls,
```

```
                do_adv_batch & do_cls,
                do_mse=self.zinb_and_mse,
            )
            losses.update({"gen_" + k: v for k, v in l.items()})
            total_loss += tloss

        # TASK 7. next time point prediction
        if do_next_tp:
            pass
        # TASK 4. contrastive cell embedding
        if do_cce:
            loss_cce = 0
            n_pairs = 0
            for i, cell_emb1 in enumerate(cell_embs[:-1]):
                for cell_emb2 in cell_embs[(i + 1) :]:
                    loss_cce += loss.contrastive_loss(
                        cell_emb1, cell_emb2, cce_temp
                    )  # (nlabels, minibatch, minibatch)
                    n_pairs += 1
            avg_loss_cce = loss_cce / max(n_pairs, 1)
            total_loss += avg_loss_cce * self.cce_scale
            # TASK 3b. contrastive graph embedding
            losses.update({"cce": avg_loss_cce})

        # TASK 8. KO profile prediction
        # if we have that information
        # TASK 9. PDgrapher-drug-like perturbation prediction (L1000?)
        return total_loss, losses

    def _compute_loss(
        self,
        output,
        expression,
        clss,
        batch_idx,
        do_ecs=False,
        do_adv_cls=False,
        do_adv_batch=False,
        do_mse=0,
    ):
        """
        _compute_loss compute the loss of the model given output from the forward pass

        Args:
            output (dict): A dictionary containing the output of the forward pass.
            expression (Tensor): A tensor containing the expression levels of genes.
            mask (Tensor): A tensor indicating the masked positions in the input data.
            clss (Tensor): A tensor containing the class classes for each cell.
            do_ecs (bool, optional): A flag to indicate whether to perform elastic cell similarity.
                Defaults to False.
            do_adv_cls (bool, optional): A flag to indicate whether to perform adversarial classification.
                Defaults to False.
            do_mse (float, optional): A scaling factor to indicate whether and how much to weight mean
            squared error loss in addition to zinb loss.
                Defaults to 0.

        Raises:
            ValueError: Raised when an invalid operation or input is encountered.

        Returns:
            tuple: A tuple containing the total loss as a float and the individual losses as a dictionary.
        """
        total_loss = 0
        losses = {}
        # TASK 1. reconstruct masked expression
        if "zero_logits" in output:
            loss_expr = loss.zinb(
                theta=output["disp"],
                pi=output["zero_logits"],
                mu=output["mean"],
                target=expression,
            )
            if do_mse:
                loss_expr += (
                    loss.mse(
```

```
                        input=torch.log(output["mean"] + 1)
                        * (1 - torch.sigmoid(output["zero_logits"])),
                        target=torch.log(expression + 1),
                    )
                    / 10  # scale to make it more similar to the zinb
                )
        elif "disp" in output:
            loss_expr = loss.nb(
                theta=output["disp"],
                mu=output["mean"],
                target=expression,
            )
        elif "mean" in output:
            loss_expr = loss.mse(
                input=output["mean"],
                target=expression,
            )
        else:
            loss_expr = 0
        total_loss += loss_expr
        losses.update({"expr": loss_expr})

        # TASK 2. predict classes
        if len(self.classes) > 0 and "cell_embs" in output:
            ## Calculate pairwise cosine similarity for the embeddings
            # Calculate pairwise cosine similarity more efficiently
            loss_emb_indep = loss.within_sample(output["cell_embs"])
            losses.update({"emb_independence": loss_emb_indep})
            total_loss += self.class_embd_diss_scale * loss_emb_indep
            ## compute class loss
            loss_cls = 0
            loss_adv_cls = 0
            for j, clsname in enumerate(self.classes):
                if "cls_output_" + clsname not in output:
                    continue
                # setting the classes from index to one hot
                loss_cls += loss.classification(
                    clsname,
                    pred=output["cls_output_" + clsname],
                    cl=clss[:, j],
                    maxsize=self.label_counts[clsname],
                    labels_hierarchy=self.mat_labels_hierarchy,
                )
            total_loss += self.class_scale * loss_cls
            if loss_cls != 0:
                losses.update({"cls": loss_cls})
            # TASK 2bis. adversarial label prediction
            if do_adv_cls:
                embs = output["cell_embs"][
                    :, (2 if self.depth_atinput else 1) :, :
                ].clone()
                for j, adv_cls in enumerate(self.classes):
                    ind = torch.arange(len(self.classes))
                    mean_embs = torch.mean(embs[:, ind != j, :], dim=1)
                    mean_embs = grad_reverse(mean_embs, lambd=1.0)
                    adv_pred = self.cls_decoders[adv_cls](mean_embs)
                    loss_adv_cls += loss.classification(
                        adv_cls,
                        pred=adv_pred,
                        cl=clss[:, j],
                        maxsize=self.label_counts[adv_cls],
                        labels_hierarchy=self.mat_labels_hierarchy,
                    )

                total_loss += self.adv_class_scale * loss_adv_cls
                losses.update({"adv_cls": loss_adv_cls})

        if (
            do_adv_batch
            and self.grad_reverse_discriminator_loss is not None
            and batch_idx is not None
            and "cell_embs" in output
        ):
            mean_emb = torch.mean(
                output["cell_embs"][:, (2 if self.depth_atinput else 1) :, :].clone(),
```

```
                dim=1,
            )
            loss_adv = self.grad_reverse_discriminator_loss(mean_emb, batch_idx)
            total_loss += loss_adv * self.class_scale / 16
            losses.update({"adv_batch": loss_adv})
        # TASK 2ter. cell KO effect prediction
        # (just use a novel class, cell state and predict if cell death or not from it)
        # add large timepoint and set the KO gene to a KO embedding instead of expression embedding
        # TODO: try to require the gene id to still be predictable (with weight tying)
        if "mvc_zero_logits" in output:
            loss_expr_mvc = loss.zinb(
                theta=output["mvc_disp"],
                pi=output["mvc_zero_logits"],
                mu=output["mvc_mean"],
                target=expression,
            )
            total_loss += loss_expr_mvc * self.mvc_scale
            losses.update({"expr_mvc": loss_expr_mvc})
        elif "mvc_mean" in output:
            loss_expr_mvc = loss.mse(
                input=output["mvc_mean"],
                target=expression,
            )
            total_loss += loss_expr_mvc * self.mvc_scale
            losses.update({"expr_mvc": loss_expr_mvc})
        # TASK 5. elastic cell similarity
        if do_ecs and "cell_emb" in output:
            loss_ecs = loss.ecs(output["cell_emb"], ecs_threshold=self.ecs_threshold)
            total_loss += self.ecs_scale * loss_ecs
            losses.update({"ecs": loss_ecs})
        return losses, total_loss

    def optimizer_step(self, epoch, batch_idx, optimizer, optimizer_closure):
        """@see pl.LightningModule"""
        # update params
        # manually warm up lr without a scheduler
        # making sure that we don't do this during lrfinder
        lr_scale = None
        prev_lr = None
        if (
            self.trainer.global_step < self.warmup_duration + self.lrfinder_steps
        ) and self.lrfinder_steps <= self.trainer.global_step:
            for i, pg in enumerate(optimizer.param_groups):
                lr_scale = min(
                    1.0, float(self.trainer.global_step + 1) / self.warmup_duration
                )
                prev_lr = pg["lr"]
                pg["lr"] = lr_scale * self.hparams.lr
        for i, pg in enumerate(optimizer.param_groups):
            # if pg["lr"] < 2e-5:
            #    pg["lr"] = 2e-5
            self.log("lr_" + str(i), pg["lr"])
        if optimizer.param_groups[0]["lr"] > self.hparams.lr:
            print(optimizer.param_groups[0]["lr"], self.hparams.lr)
            print(lr_scale, self.warmup_duration, self.trainer.global_step, prev_lr)
            if prev_lr is not None:
                pg["lr"] = prev_lr
            else:
                raise ValueError("OPTIMIZER HAS INCREASED LR. WHYY?")

        optimizer.step(closure=optimizer_closure)

    def on_validation_start(self):
        for k, v in self.mat_labels_hierarchy.items():
            self.mat_labels_hierarchy[k] = v.to(self.device)

    def on_validation_epoch_start(self):
        self.embs = None
        self.counter = 0

    def validation_step(
        self,
        batch,
        batch_idx,
    ):
```

```python
        """
        validation_step defines the validation loop. It is independent of forward
        @see pl.LightningModule

        Args:
            batch (list[Tensor]): @see training_step
        """
        val_loss, losses = self._full_training(
            batch=batch,
            do_denoise=self.do_denoise,
            noise=self.noise,
            do_next_tp=self.do_next_tp,
            do_cce=self.do_cce,
            cce_temp=self.cce_temp,
            do_ecs=self.do_ecs,
            do_mvc=self.do_mvc,
            do_adv_cls=self.do_adv_cls,
            do_adv_batch=self.do_adv_batch,
            do_cls=self.do_cls,
            do_generate=self.do_generate,
            run_full_forward=self.run_full_forward,
            mask_ratio=self.mask_ratio,
        )
        expression = batch["x"]
        gene_pos = batch["genes"]
        depth = batch["depth"]
        # TODO: make this faster by only calling val loss
        if self.embs is not None:
            if self.embs.shape[0] < 100_000:
                self.info = torch.cat([self.info, batch["class"]])
                self._predict(
                    gene_pos,
                    expression,
                    depth,
                    pred_embedding=self.pred_embedding,
                    max_size_in_mem=120_000,
                )
        else:
            self.info = batch["class"]
            self._predict(
                gene_pos,
                expression,
                depth,
                pred_embedding=self.pred_embedding,
                max_size_in_mem=120_000,
            )
        self.log("val_loss", val_loss, sync_dist=True)
        self.log_dict(losses, sync_dist=True)
        return val_loss

    def on_validation_epoch_end(self):
        """@see pl.LightningModule"""
        self.embs = self.all_gather(self.embs).view(-1, self.embs.shape[-1])
        self.info = self.all_gather(self.info).view(-1, self.info.shape[-1])
        self.pred = (
            self.all_gather(self.pred).view(-1, self.pred.shape[-1])
            if self.pred is not None
            else None
        )
        self.pos = self.all_gather(self.pos).view(-1, self.pos.shape[-1])
        if self.trainer.state.stage != "sanity_check":
            if self.trainer.is_global_zero:
                print("logging anndata")
                sch = self.lr_schedulers()
                sch.step(self.trainer.callback_metrics["val_loss"])
                # run the test function on specific dataset
                self.log_adata(
                    gtclass=self.info, name="validation_part_" + str(self.counter)
                )
                if (self.current_epoch + 1) % self.test_every == 0:
                    self.on_test_epoch_end()
                # Synchronize all processes with a timeout
            if torch.distributed.is_initialized():
                # Set a timeout that's longer than your test typically takes
                # Write rank to file for debugging
```

```
                self.trainer.strategy.barrier()

    def test_step(self, *args, **kwargs):
        pass

    def on_test_epoch_end(self):
        # Run the test only on global rank 0
        name = self.name + "_step" + str(self.global_step)
        try:
            metrics = utils.test(self, name, filedir=str(FILEDIR), do_class=self.do_cls)
            print(metrics)
            print("done test")
            self.log_dict(metrics, sync_dist=False, rank_zero_only=True)
        except Exception as e:
            import traceback

            print(f"Error during test: {e}")
            print("Full traceback:")
            print(traceback.format_exc())
            print("Skipping test metrics logging")

    def on_predict_epoch_start(self):
        """@see pl.LightningModule"""
        self.embs = None
        self.attn.data = None
        self.attn.attn = None
        self.counter = 0
        if type(self.transformer) is FlashTransformer:
            for encoder_layers in self.transformer.blocks:
                encoder_layers.set_seq_parallel(False)

    def predict_step(self, batch, batch_idx):
        """
        embed given gene expression, encode the gene embedding and cell embedding.

        Args:
            batch @see training_step

        Returns:
            Tensor: _description_
        """
        return self._predict(
            batch["genes"],
            batch["x"],
            batch["depth"],
            self.predict_mode,
            self.pred_embedding,
            self.get_attention_layer,
            self.predict_depth_mult,
        )

    def _predict(
        self,
        gene_pos,
        expression,
        depth,
        predict_mode="none",
        pred_embedding=[],
        get_attention_layer=[],
        depth_mult=6,
        keep_output=True,
        max_size_in_mem=100_000,
        get_gene_emb=False,
    ):
        """
        @see predict_step will save output of predict in multiple self variables

        - embs: the cell embeddings (means from label specific embeddings given by self.pred_embedding)
        - pred: the predicted cell classes
        - pos: the genes used
        - expr_pred: the expression prediction. [mean, disp, zero_logits]
        - mean_attn: the mean attention across cells for the given layer (in self.get_attention_layer)

        these will be finalized in self.on_predict_epoch_end()

```

```
        Args:
            @see training_step
            other important arguments:
            keep_output (bool, optional): whether to keep the output in memory. Defaults to True.
            self.get_attention_layer (list, optional): the layers to get the attention from. Defaults to [].
            self.pred_embedding (list, optional): the classes to predict. Defaults to [].

        """
        if predict_mode == "none":
            output = self.forward(
                gene_pos,
                expression,
                depth_mult=expression.sum(1),
                req_depth=depth,
                get_attention_layer=get_attention_layer,
                do_class=True,
                get_gene_emb=get_gene_emb,
            )
            if len(get_attention_layer) > 0:
                self.attn.add([i[:, :, :2, :] for i in output[1]], gene_pos)
                output = output[0]
            cell_embs = output["cell_embs"]
        elif predict_mode == "denoise":
            output = self.forward(
                gene_pos,
                expression,
                depth_mult=expression.sum(1) * depth_mult,
                req_depth=depth * depth_mult,
                get_attention_layer=get_attention_layer,
                do_class=True,
                get_gene_emb=get_gene_emb,
            )
            if len(get_attention_layer) > 0:
                self.attn.add([i[:, :, :2, :] for i in output[1]], gene_pos)
                output = output[0]
            cell_embs = output["cell_embs"]
        elif predict_mode == "generate":
            output = self.forward(
                gene_pos,
                expression,
                req_depth=depth,
                do_mvc=False,
                do_class=False,
                get_gene_emb=get_gene_emb,
            )
            cell_embs = output["cell_embs"]
            output = self._generate(
                output["cell_embs"],
                gene_pos,
                req_depth=None,  # otherwise we have 2 depths passed
                depth_mult=expression.sum(1),
                do_class=self.do_cls,
                do_mvc=False,
            )
        else:
            raise ValueError(
                "predict_mode needs to be one of ['none', 'denoise', 'generate']"
            )

        if len(pred_embedding) == 0:
            pred_embedding = self.classes
        ind = [
            self.classes.index(i) + (2 if self.depth_atinput else 1)
            for i in pred_embedding
        ]
        if not keep_output:
            return {
                "embs": torch.mean(cell_embs[:, ind, :], dim=1),
                "class": (
                    torch.stack(
                        [
                            torch.argmax(output["cls_output_" + clsname], dim=1)
                            for clsname in self.classes
                        ]
                    ).transpose(0, 1)
```

```
                    if len(self.classes) > 0
                    else None
                ),
                "pos": gene_pos,
                "expr": (
                    [output["mean"], output["disp"], output["zero_logits"]]
                    if "disp" in output
                    else [output["mean"]]
                ),
            }
        if self.embs is None:
            self.embs = torch.mean(cell_embs[:, ind, :], dim=1)
            # self.embs = output["cls_output_" + "cell_type_ontology_term_id"]
            self.pred = (
                torch.stack(
                    [
                        (
                            torch.argmax(output["cls_output_" + clsname], dim=1)
                            if not self.keep_all_cls_pred
                            else output["cls_output_" + clsname]
                        )
                        for clsname in self.classes
                    ]
                ).transpose(0, 1)
                if len(self.classes) > 0
                else None
            )
            self.pos = gene_pos
            self.expr_pred = (
                [output["mean"], output["disp"], output["zero_logits"]]
                if "disp" in output
                else [output["mean"]]
            )
        else:
            self.embs = torch.cat(
                # [self.embs, output["cls_output_" + "cell_type_ontology_term_id"]]
                [self.embs, torch.mean(cell_embs[:, ind, :], dim=1)]
            )
            self.pred = torch.cat(
                [
                    self.pred,
                    (
                        torch.stack(
                            [
                                (
                                    torch.argmax(output["cls_output_" + clsname], dim=1)
                                    if not self.keep_all_cls_pred
                                    else output["cls_output_" + clsname]
                                )
                                for clsname in self.classes
                            ]
                        ).transpose(0, 1)
                        if len(self.classes) > 0
                        else None
                    ),
                ],
            )
            self.pos = torch.cat([self.pos, gene_pos])
            self.expr_pred = (
                [
                    torch.cat([self.expr_pred[0], output["mean"]]),
                    torch.cat([self.expr_pred[1], output["disp"]]),
                    torch.cat([self.expr_pred[2], output["zero_logits"]]),
                ]
                if "disp" in output
                else [torch.cat([self.expr_pred[0], output["mean"]])]
            )
        if self.embs is not None:
            if self.embs.shape[0] > max_size_in_mem and self.pred_log_adata:
                print("logging")
                self.log_adata(name="predict_part_" + str(self.counter))
                self.counter += 1
                self.pos = None
                self.expr_pred = None
                self.pred = None
```

```
                self.embs = None
            elif not self.pred_log_adata:
                print(
                    "WARNING, reached max size in memory, deleting the adata, \
                    need to set pred_log_adata to True to log the adata"
                )

    def on_predict_epoch_end(self):
        """@see pl.LightningModule will"""
        if self.pos.shape[0] < 100:
            return
        if self.pred_log_adata:
            print("adding on disk")
            return self.log_adata(name="predict_part_" + str(self.counter))

    def get_cell_embs(self, layer_output):
        """
        get_cell_embs

        Args:
            layer_output (Tensor): The output tensor from a layer in the model.

        Raises:
            ValueError: Raised when an unknown cell embedding style is encountered.

        Returns:
            Tensor: The cell embeddings tensor.
        """
        if self.cell_emb_style == "cls" and self.classes is not None:
            # (minibatch, embsize)
            cell_emb = layer_output[:, : self.cell_embs_count]
        elif self.cell_emb_style == "avg-pool":
            cell_emb = torch.mean(layer_output, dim=1)
        else:
            raise ValueError(f"Unknown cell_emb_style: {self.cell_emb_style}")
        return cell_emb

    def _generate(
        self,
        cell_embs: Tensor,
        gene_pos: Tensor,
        depth_mult: Tensor,
        req_depth: Optional[Tensor] = None,
        **decoder_kwargs,
    ):
        """
        _generate given cell_embeddings, generate an expression profile

        the goal was to iterate multiple times,
        to create a trajectory and reach a certain state
        should call forward multiple times

        Args:
            cell_emb(:obj:`Tensor`): A tensor representing cell embeddings. It has a shape of (minibatch, embsize).
            src(:obj:`Tensor`): A tensor representing the source data. It has a shape of (minibatch, seq_len).
            values(:obj:`Tensor`): An optional tensor representing the values. It has a shape of (minibatch, seq_len).
            gen_iters(:obj:`int`): An integer representing the number of generation iterations.
            classes(:obj:`Tensor`): An optional tensor representing the classes. It has a shape of (batch,).
        """
        if req_depth is not None and self.depth_atinput:
            cell_embs = torch.cat([cell_embs[:, :1, :], cell_embs[:, 2:, :]], dim=1)
        encoding = self._encoder(
            cell_embs=cell_embs,
            gene_pos=gene_pos,
            req_depth=req_depth if self.depth_atinput else None,
        )
        if self.cell_transformer:
            gene_encoding = encoding[:, self.cell_embs_count :, :]
            cell_embs = encoding[:, : self.cell_embs_count, :]
            transformer_output = self.transformer(gene_encoding, x_kv=cell_embs)
            transformer_output = torch.cat([cell_embs, transformer_output], dim=1)
        else:
            transformer_output = self.transformer(encoding)
        output = self._decoder(
```

```python
            transformer_output,
            depth_mult=depth_mult,
            req_depth=req_depth if not self.depth_atinput else None,
            **decoder_kwargs,
        )
        if self.cell_transformer:
            output.pop("cell_embs")
            output.pop("cell_emb")
        return output  # (minibatch, seq_len)

    def log_adata(self, gtclass=None, name=""):
        """
        log_adata will log an adata from predictions.
        It will log to tensorboard and wandb if available

        see @utils.log_adata
        """
        try:
            mdir = self.logger.save_dir if self.logger.save_dir is not None else "/tmp"
        except:
            mdir = "data/"
        if not os.path.exists(mdir):
            os.makedirs(mdir)
        adata, fig = utils.make_adata(
            self.embs,
            self.classes,
            self.pred if not self.keep_all_cls_pred else None,
            self.attn.get(),
            self.global_step,
            self.label_decoders,
            self.labels_hierarchy,
            gtclass,
            self.name + "_" + name + "_" + str(self.global_rank),
            mdir,
            self.doplot,
        )
        if self.doplot:
            try:
                self.logger.experiment.add_figure(fig)
            except:
                print("couldn't log to tensorboard")
            try:
                self.logger.log_image(key="umaps", images=[fig])
            except:
                print("couldn't log to wandb")

        return adata

    def _predict_denoised_expression(self, gene_pos, expression, depth):
        """
        Args:
            gene_pos (:obj:`Tensor`): token ids, shape [batch_size, seq_len]
            expression (:obj:`Tensor`): token values, shape [batch_size, seq_len]

        Returns:
            dict of output Tensors.
        """
        output = self.forward(gene_pos, expression, req_depth=depth)
        return output

```